Informe de seguridad

Tienda en Línea Modular (Taller Nogal)

Materia: Desarrollo de Aplicaciones Web (UPB) · **Laboratorio:** Seguridad (Django)

Integrantes del equipo: Santiago Anaya

Rama de GitHub: `seguridad` (fusionada en `main`) · **Stack:** Django 6.1.1 + DRF 3.18.2 + PostgreSQL (Neon) en Render; frontend Vite en Vercel

Numeración OWASP: Top 10 (2025), la de la guía del laboratorio.

1. Diagnóstico (Parte 0)

Estado de la rama `seguridad` al revisarla con búsqueda global del editor (`Ctrl+Shift+F`) y con comandos. No se anotó el número de advertencias de `check --deploy` antes de los cambios; el resultado final está en la sección 5.

Pregunta ⚠	Resultado	Cómo se comprobó
¿Contraseñas guardadas tal cual?	No	Django usa PBKDF2 por defecto; la prueba `la_clave_no_se_guarda_en_claro` está en `test_cuentas.py`
¿Credenciales en el código?	No	`SECRET_KEY` y `DATABASE_URL` salen del entorno; con `DEBUG=False` la app no arranca con clave insegura
¿Archivos de secretos en git?	No	`git ls-files` solo muestra `.env.example`
¿Texto de usuarios mostrado como HTML?	No	Sin `innerHTML`, `insertAdjacentHTML`, `mark_safe`, `autoescape off` ni `@csrf_exempt` en el código propio; el frontend usa `textContent`
¿Se ve un recurso ajeno cambiando el id?	No	El carrito se identifica con UUID aleatorio; token desconocido → 404; no hay endpoint de pedidos por id
¿Rutas de admin solo ocultas en el frontend?	No	El servidor exige `is_staff` en escritura (`test_autenticacion.py`, `PermisosTests`)
¿CORS abierto a cualquier origen?	No	Solo los orígenes de `FRONTEND_ORIGINS`
¿HTTP simple?	No	En producción http → https (301)
`DEBUG`, `ALLOWED_HOSTS`, `SECRET_KEY`	Bien	`DEBUG` por defecto `False`; hosts por variable; clave por variable

2. Hallazgos y correcciones

#	Hallazgo	OWASP	Evidencia (cómo lo comprobé)	Corrección aplicada	Estado
1	El login no limitaba intentos	A07 Fallas de autenticación	Sin límite, el 6.º intento fallido seguía dando 401. Ahora: 5 intentos → 401 y el 6.º → **429** (prueba automática, servidor real con gunicorn y captura en el navegador)	Límite de 5 fallos por usuario en 15 min, `Retry-After: 900`, reinicio al acertar (`catalogo/seguridad.py`, `autenticacion.py`)	Corregido
2	Errores 404/500 podían mostrar páginas HTML de Django	A10 Condiciones excepcionales	Ruta inexistente en el backend de Render → `{"detail": "No encontrado."}` (captura)	`handler404/handler500` devuelven JSON genérico (`config/urls.py`)	Corregido
3	No había registro de eventos de seguridad	A09 Registros y alertas	Logs del servidor con correo enmascarado (`sa***@correo.com`)	`LOGGING` en `settings.py`; se registran login fallido (“intento N de 5”), bloqueo y login correcto (id y rol). Nunca contraseñas	Corregido
4	DRF 3.18.1 con vulnerabilidades conocidas	A03 Cadena de suministro	`pip-audit` reportó CVE; tras actualizar: *No known vulnerabilities found*	`djangorestframework==3.18.2`	Corregido
5	Mensaje de login distinto si el usuario existe o no	A07	Prueba: usuario inexistente y clave mala dan el mismo 401	Ya era genérico: “Usuario o contraseña incorrectos.”	Ya cubierto
6	CSP y cabeceras del frontend	A02 Configuración insegura	`vercel.json` define CSP (`script-src 'self'`, `frame-ancestors 'none'` …), `nosniff`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`. Captura: en la	Ya aplicada	Ya cubierto

			consola, un `<img onerror=...>` inyectado queda bloqueado por la CSP		
7	Escritura sin token CSRF	A01 Control de acceso roto	Con sesión admin y sin token: `fetch POST /api/categorias` → **403 "CSRF Failed: CSRF token missing."** (captura); además `CsrfTests`	Token CSRF en cabecera `X-CSRFToken`, sin `@csrf_exempt`	Ya cubierto
8	Cookies de sesión	A04 Fallas criptográficas	Captura de Application → Cookies: `sessionid` HttpOnly + Secure + SameSite Lax; `csrftoken` Secure + Lax; `document.cookie` solo devuelve `csrftoken`	Configuración de cookies en `settings.py`	Ya cubierto

3. Cambios de esta rama

Archivo	Cambio
`catalogo/seguridad.py` (nuevo)	Contador de fallos por usuario en caché (SHA-256 del usuario en minúsculas), bloqueo, limpieza y enmascarado para logs
`catalogo/autenticacion.py`	`LoginView`: 429 + `Retry-After`, registro de fallos y aciertos
`config/settings.py`	`LOGIN_MAX_INTENTOS` (5) y `LOGIN_VENTANA_SEGUNDOS` (900) por entorno; caché `LocMem`; `LOGGING`
`config/urls.py`	`handler404` y `handler500` en JSON
`catalogo/test_laboratorio_seguridad.py` (nuevo)	21 pruebas
`requirements.txt`	DRF 3.18.1 → 3.18.2

Decisión de diseño — límite por usuario y no por IP: el frontend llega al backend a través del proxy de Vercel, así que todas las peticiones parecen venir de la misma IP. Contrapartida: alguien podría bloquear 15 minutos la cuenta de otra persona. Limitación: la caché es por proceso; sirve porque Render (plan gratuito) corre un solo worker de gunicorn. Con varios workers habría que usar Redis o la base de datos.

4. Preguntas del laboratorio

¿Qué dos garantías da HTTPS que HTTP no da? ¿Garantiza que el sitio sea honesto? HTTPS cifra el tráfico (nadie en el camino puede leerlo o alterarlo) y autentica al servidor mediante el certificado (el navegador comprueba que realmente habla con ese dominio). No garantiza que el sitio sea honesto: un sitio fraudulento también puede tener un certificado válido.

¿Por qué alguien pondría `@csrf_exempt` "porque da error 403" y por qué es mala idea? Porque quita el error sin entender su causa. Pero el 403 es la protección funcionando: sin ella, cualquier página externa podría hacer que el navegador de una persona con sesión abierta envíe peticiones que modifican datos. Lo correcto es enviar el token en la cabecera. Este proyecto no usa `@csrf_exempt` en ninguna vista (la compra anónima no lo necesita porque no usa sesión).

Si se usa SameSite, ¿por qué se añade además el token? Defensa en profundidad: SameSite depende del navegador (los antiguos no lo respetan), no protege entre subdominios del mismo sitio y `Lax` sí envía la cookie en navegaciones de nivel superior. El token se verifica en el servidor, que es la parte que controlamos. Este proyecto usa `SameSite=Lax`.

5. Verificaciones ejecutadas

Comprobación	Resultado
`python manage.py test`	150 pruebas, OK
`pip-audit -r requirements.txt`	No known vulnerabilities found
`python manage.py check --deploy` (con `SECRET_KEY` larga y `DEBUG=False`)	Sin advertencias propias del proyecto (la W009 solo sale si la clave es corta)
Servidor real con gunicorn	301 http→https; cabeceras de seguridad; 404 JSON; 5 fallos → 401, 6.º → 429 con `Retry-After: 900`; logs con correo enmascarado
Producción (Render)	La ruta inexistente responde el JSON genérico

6. Pruebas de diseño inseguro (A06)

Prueba	Resultado	Estado
Cantidad negativa o cero al agregar o cambiar	400; el total no se altera	Corregido / verificado (prueba automática)
Precio o total manipulado en el cuerpo	Se ignora; el servidor lee el precio de la base de datos	Verificado (prueba automática)
`rol: admin` (o `is_staff`) al registrarse	Se ignora; el usuario queda como cliente	Verificado (prueba automática)

7. Auditoría OWASP Top 10 (2025)

OWASP	Pregunta de revisión	Resultado	Dónde se comprobó
A01 Control de acceso roto	¿Cada ruta verifica rol y propietario en el servidor?	Sí: escritura solo `is_staff`; carrito por UUID; CSRF	`PermisosTests`, `CsrfTests`
A02 Configuración insegura	¿CORS, DEBUG y cabeceras están bien?	Sí: `DEBUG=False`, CORS por lista, HSTS, CSP	`test_seguridad.py`, `vercel.json`, `check --deploy`
A03 Cadena de suministro	¿Se auditaron las dependencias?	Sí, `pip-audit` limpio. Falta firma/verificación de artefactos	§5
A04 Fallas criptográficas	¿HTTPS? ¿Hash lento? ¿Secretos fuera del código?	Sí: HTTPS + HSTS, PBKDF2, secretos por entorno	`test_seguridad.py`, `test_cuentas.py`
A05 Inyección (incluye XSS)	¿Salida escapada y entradas validadas?	Sí: ORM parametrizado, `textContent`, CSP, validación de URLs de imagen	`test_compra.py`, captura CSP
A06 Diseño inseguro	¿Hay lógica abusable?	No: total y envío calculados en el servidor	§6
A07 Fallas de autenticación	¿Límite de intentos y mensajes genéricos?	Sí. La sesión caduca con el valor por defecto de Django (2 semanas); mejora futura: acortar la duración de la sesión	`LimiteDeIntentosTests`
A08 Integridad de software o datos	¿Se confía en datos del cliente?	No: el rol y el precio los decide el servidor; la sesión es del servidor	§6
A09 Registros y alertas	¿Se registran los logins fallidos y los errores?	Sí en los logs de Render; no hay alertas automáticas	`RegistrosDeSeguridadTests`
A10 Condiciones excepcionales	¿Los errores fallan de forma segura?	Sí: 404/500 genéricos; login bloqueado niega aunque la clave sea correcta	`ErroresGenericosTests`

8. Matriz de riesgos

#	Riesgo	Probabilidad	Impacto	Tratamiento
R1	Fuga de la cadena de conexión de Neon (`DATABASE_URL`)	Media	Alto	Mitigar: solo en variables de Render, nunca en git; rotar si se expone (§9)
R2	Fuerza bruta contra el login	Media	Medio	Mitigado: 429 tras 5 fallos; revisar los logs
R3	Bloqueo intencional de una cuenta ajena	Baja	Bajo	Aceptar: dura 15 minutos
R4	Dependencia con vulnerabilidad nueva	Media	Medio	Mitigar: `pip-audit` periódico y antes de cada entrega
R5	XSS por contenido del catálogo	Baja	Alto	Mitigado: `textContent` + CSP
R6	La app se conecta a Neon con el rol dueño (`neondb_owner`, el único rol) y la base acepta conexiones desde cualquier IP (la lista de IP permitidas no está disponible en el plan gratuito)	Baja	Alto	Aceptar por ahora y mitigar con la contraseña fuera del código y rotación (§9). Mejora futura: rol con permisos mínimos para la app (solo `SELECT/INSERT/UPDATE/DELETE`) y otro para `migrate`; lista de IP permitidas si se pasa a un plan que la ofrezca

9. Plan ante un incidente (NIST) — fuga de la cadena de conexión de Neon

1. **Preparación:** secretos solo en variables de entorno de Render; `.env` ignorado; copia de seguridad en Neon.
2. **Detección y análisis:** revisar si aparece en el repositorio (`git log -S "postgres"`), la actividad inusual en el panel de Neon y los logs de Render; fijar desde cuándo estuvo expuesta.

3. **Contención:** rotar de inmediato la contraseña del rol en Neon (o crear otro rol y revocar el anterior); actualizar `DATABASE_URL` en Render y redesplegar.
4. **Erradicación:** si estuvo en git, quitarla del historial y asumir que está comprometida aunque se borre; buscar otras copias (capturas, chats, documentos).
5. **Recuperación:** comprobar la tienda (catálogo, registro, login, compra), revisar que no haya filas o usuarios extraños y restaurar desde copia si hubo alteraciones.
6. **Lecciones aprendidas:** documentar causa y tiempo de detección; revisar secretos antes de cada commit.

10. Datos personales (Ley 1581 de 2012)

- **Cuentas registradas:** nombre, correo (también sirve de usuario) y contraseña con hash. Finalidad: iniciar sesión y reconocer al cliente.
- **Compra simulada:** pide nombre, correo, teléfono, dirección y ciudad, y los devuelve en el resumen; **el backend no guarda pedidos** (no existe modelo de pedido; solo `Carrito` e `ItemCarrito`). No se piden datos de pago.
- **Qué podría sobrar:** el teléfono, si la entrega no depende de él; sería razonable hacerlo opcional (mejora futura).
- **Qué falta para un uso real:** política de tratamiento de datos visible, autorización expresa del titular antes de enviar el formulario y un procedimiento para consultar, corregir o eliminar sus datos.

11. Cierre

1. **¿Cuál fue el hallazgo más grave y por qué?** El login sin límite de intentos. Era lo más fácil de aprovechar: cualquiera podía probar contraseñas sin freno hasta acertar, y la cuenta de administrador controla el catálogo. Lo corregí con un bloqueo de 5 intentos fallidos por usuario durante 15 minutos (respuesta 429) y con registro de cada fallo. La contrapartida es que alguien podría bloquear a propósito una cuenta ajena por unos minutos.
2. **¿Qué defensa habría evitado el caso de la clase que más me impactó?** El caso de Uber de 2022. Según lo reportado, el atacante consiguió las credenciales de un contratista, le envió muchas notificaciones de aprobación de acceso (fatiga de MFA) hasta que una fue aceptada, y luego encontró en un script credenciales de administrador escritas en claro, con las que llegó a muchos sistemas. Habrían ayudado un MFA resistente a ese engaño (por ejemplo, con número de verificación o llave física), no dejar credenciales dentro de scripts o código y límites de intentos con alertas. En este proyecto aplicamos lo segundo (`SECRET_KEY` y `DATABASE_URL` solo en variables de entorno, `.env` fuera de git) y lo tercero (429 y registros).
3. **Mejoras futuras:** pruebas de seguridad automáticas en cada push (las 150 pruebas más `pip-audit`, por ejemplo con GitHub Actions) para detectar a tiempo una dependencia vulnerable o un cambio que debilite el login, el CSRF o los permisos. También dejaría anotados MFA para el administrador y un rol con permisos mínimos en Neon.

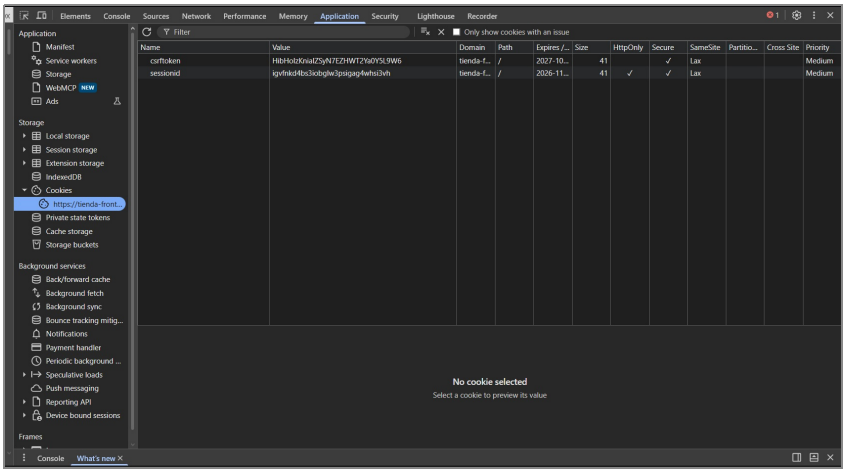
Ninguna medida alcanza sola: la seguridad es defensa en profundidad y se piensa desde el diseño.

12. Evidencia del navegador

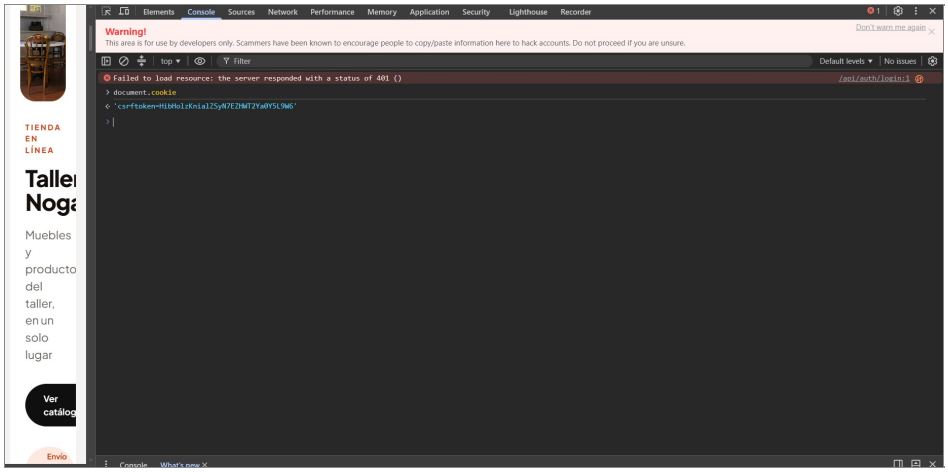
Las capturas están en la carpeta `evidencias/` (índice en `evidencias/README.md`).

- ☒Cookies: `sessionid` HttpOnly + Secure, `csrftoken` Secure; SameSite Lax.
- ☒ `document.cookie` no muestra `sessionid`.
- ☒CSP bloquea un manejador en línea inyectado.
- ☒CSRF: sesión admin sin token → 403 “CSRF token missing”. *Nota:* la prueba con `ataque.html` devolvió el 404 de Vercel (no llegó al backend); se reemplazó por esta petición equivalente.
- ☒Login: el 6.º intento muestra “Demasiados intentos. Espere 15 minutos…”.
- ☒Revisión de Neon (proyecto `tienda-modular`, rama `production`): un solo rol, `neondb_owner`, dueño de la base; acceso público a internet activo con “Any IP address” (la lista de IP permitidas y el acceso por VPC requieren un plan de pago); la rama `production` figura como “Not protected”. Son límites del plan gratuito y quedan registrados como riesgo R6.
- ☒Neon → Settings → Project permissions: 1 sola persona con acceso al proyecto (el propietario del proyecto, con permiso Admin).
- No realizado: HTTPS local con mkcert (la tienda ya corre sobre HTTPS en producción) y captura del 429 en la pestaña Network.

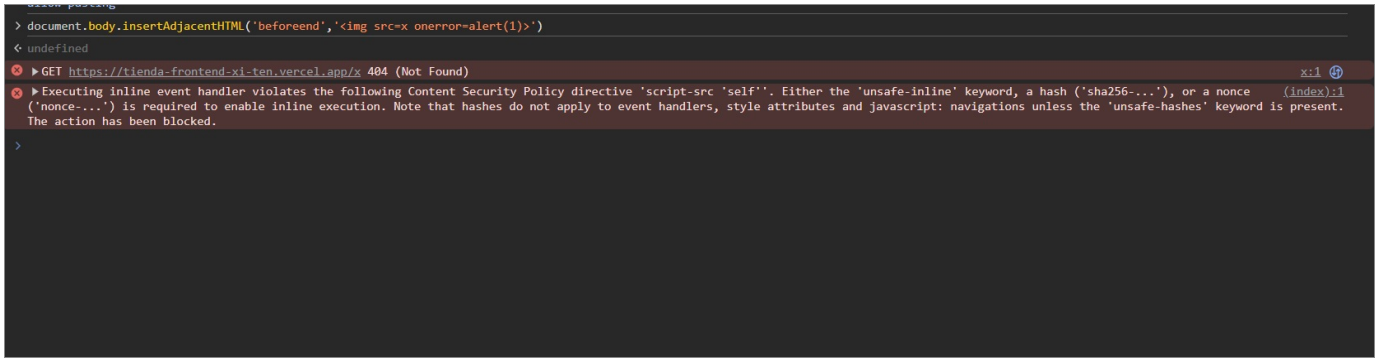
Anexo — Capturas de evidencia



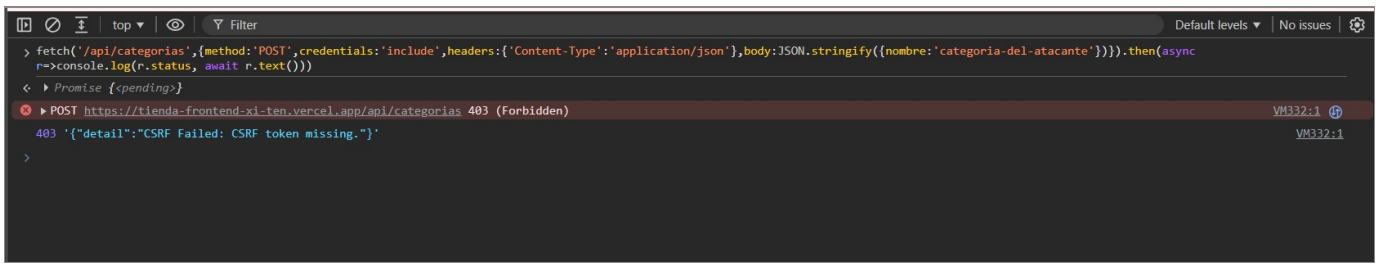
Cookies de la tienda: `sessionid` HttpOnly + Secure; `csrftoken` Secure; SameSite Lax (hallazgo 8).



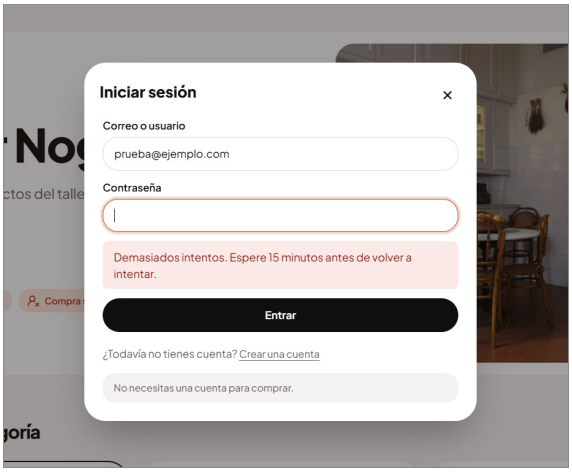
`document.cookie` solo devuelve `csrftoken`; JavaScript no puede leer la sesión (hallazgo 8).



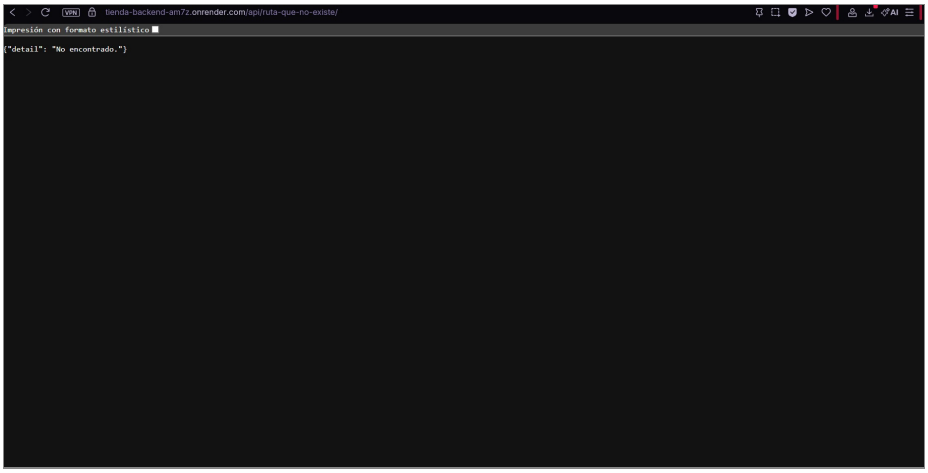
La CSP bloquea un manejador en línea inyectado (hallazgo 6).



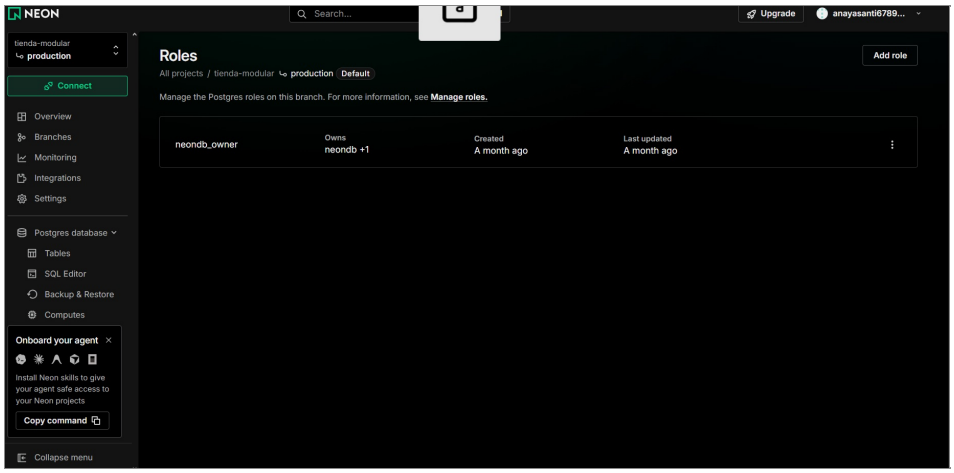
Con sesión de administrador y sin token: 403 “CSRF token missing” (hallazgo 7).



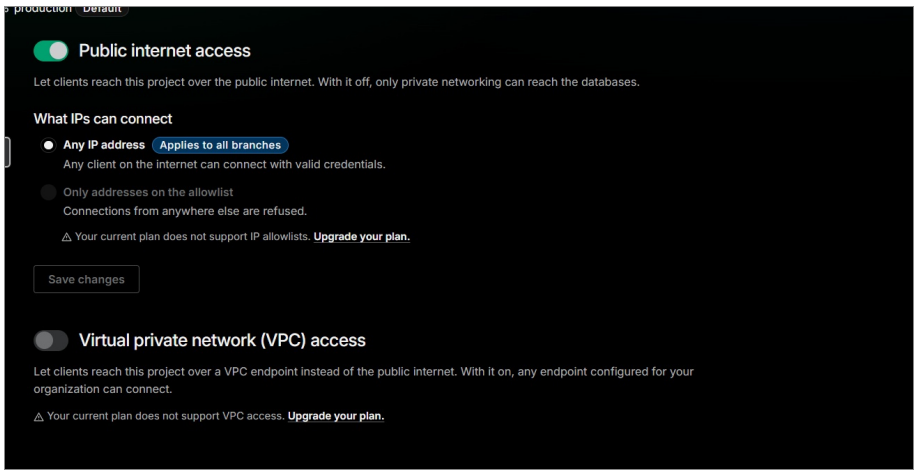
Sexto intento fallido de login: “Demasiados intentos. Espere 15 minutos...” (hallazgo 1).



Ruta inexistente en el backend de Render: JSON genérico (hallazgo 2).



Neon: un solo rol, `neondb_owner` (riesgo R6).



Neon: acceso desde cualquier IP; la lista de IP y la VPC requieren plan de pago (riesgo R6).